European Conference on Computational Biology, 2026, pp. 1–10
DOI added during production
Published: Date added during production
Paper

VCF-RDFizer: A Conversion Tool to Represent Variant Data as a Knowledge Graph

Elias Crum,[1,2,*] Bart Buelens,[1] Ruben Taelman[2] and Gökhan Ertaylan[1]

[2]IDLab, Department of Electronics and Information Systems, Ghent University – imec, Technologiepark-Zwijnaarde 116, 9052, Belgium
[1]Digital Biological Systems, Flemish institute for Technological Research (VITO), Boeretang 200, 2400, Belgium

[*]Corresponding author. E-mail: ecrum@ugent.be

Abstract

The Variant Call Format (VCF) is the de facto standard for representing human genomic variation and underpins large-scale population genomics, clinical sequencing, and precision medicine initiatives. As uses for VCF data expand, so do opportunities to enrich variant data through integration with complementary resources without modifying the VCF specification itself. Such enrichment may include additional metadata, sample- or patient-level information, test results, links to public databases, or privacy-related annotations. To accommodate the structured and scalable linkage of these heterogeneous data, we see promise in the use of a knowledge graph representation via the Resource Description Framework (RDF). To benefit from the potential of RDF, the initial challenge is translating VCF file data into RDF. Here, we present VCF-RDFizer, a command-line tool available on PyPI and Conda-Forge that provides an end-to-end pipeline for converting VCF into RDF. VCF-RDFizer introduces a publicly available vocabulary that models VCF constructs while integrating established ontologies from biomedical knowlge graphs. The tool preserves variant-level detail, genotype information, and metadata, enabling semantically an enriched and standards-aware representation of genomic variation. Unlike ad hoc or partial transformations, VCF-RDFizer offers a systematic and reproducible workflow from raw VCF to structured RDF and full coverage of all data represented in VCF files. To assess the feasibility of our tool, we tested VCF-RDFizer on six publicly available VCF files of varying sizes and complexities. Our evaluation demonstrates that conversion is tractable across realistic dataset scales and can be performed successfully on standard hardware. By transforming VCF into an interoperable knowledge graph format, VCF-RDFizer lays the foundation for the exciting possibilities of heterogeneous data enrichment of VCF.

Keywords Knowledge Representation, Genomic Mutation Data, Semantics

Introduction

The Variant Call Format (VCF) is the dominant standard for genomic variant exchange across research and clinical pipelines, including pharmacogenomics and precision medicine applications (3; 24; 29). As its adoption has expanded, downstream applications increasingly require integration of variant calls with additional contextual information, including sample- and patient-level descriptors, phenotype associations, external database references, and policy annotations governing data access. This requirement is already visible in operational semantic-genomics efforts that connect variant data to external knowledge resources and structured governance layers for controlled access (26; 32; 9). Consequently, practical analysis workflows would increasingly benefit from query models that can combine variant facts, linked biomedical context, and access constraints.

While VCF provides a compact and expressive encoding of genomic variation, it was not designed as a general-purpose semantic integration framework for linking heterogeneous data sources. These limitations become particularly apparent in domains such as privacy protection, where fine-grained and machine-interpretable access control mechanisms are desirable. In other domains, frameworks such as the Open Digital Rights Language (ODRL) (21) enable granular policy expression directly linked to data resources. Representing variant data in a form that can accommodate similar policy annotations would support controlled access and governance use cases; however, the VCF syntax offers no natural mechanism for expressing such structured constraints. More broadly, querying VCF data at scale, such as retrieving subsets of variants by genomic region or annotation criteria, typically relies on specialized command-line utilities (e.g., bcftools, htslib) or custom processing pipelines rather than declarative, interoperable query mechanisms.

Representing VCF data using the Resource Description Framework (RDF) (14) provides a principled solution to enriching VCF data and expanding its usage. RDF models data as a graph of explicitly defined entities and relationships identified by machine-readable globally unique identifiers (IRIs) (14). This representation strategy enables (i) structured linkage between variant data and external biomedical resources (8; 28), (ii) native declarative and expressive querying over graph-structured data via the SPARQL query language (19), and (iii) unique methods of fine-grained privacy policy enforcement over VCF data and linkages (21; 9). Practically, when combined, RDF

Received: Date Month Year. **Revised received:** Date Month Year. **Accepted:** Date Month Year
© The Author(s) 2026. Copyright and licence statements to be updated by the publisher during production.

representations of VCF files could enable complex, expressive queries that span variant data and linked resources, such as "find all VCF records (in a given database) whose "ALT" alleles in gene X that are annotated with phenotype Y and have access restrictions Z".

Converting VCF into RDF requires *semantic lifting*, which means transforming row-based file content into machine-interpretable graph statements. In practical terms, each relevant VCF element (e.g., variant, allele, genotype, sample) is given a stable global identifier (an IRI), and each data field is converted into RDF facts of the form subject–predicate–object (triples), where the predicate defines the relationship type. Using ontology-aligned predicates ensures that these relationships have consistent meaning across datasets, so the final RDF graph can be queried and linked reliably. This conversion process contains two important components: 1) the vocabulary and semantic schema for defining IRIs that represent particular VCF file data entries, and 2) a computational process that maps the vocabulary terms to the recorded data within a VCF file. Crucially, this conversion process must be systematic and reproducible rather than file-specific or hard-coded, particularly given the variability in VCF header declarations and field definitions.

Although VCF adheres to a formal specification (3), its structure permits substantial heterogeneity across datasets. INFO fields often contain semi-structured or multi-valued annotations; FORMAT and genotype columns may differ in composition and semantics between files; alternate allele representations extend beyond simple SNPs to complex structural variants; and metadata header lines introduce dataset-specific definitions. Multi-sample VCF files further increase structural complexity by aggregating per-sample genotype information within a shared variant record. This combination of flexibility and heterogeneity complicates the construction of a uniform semantic representation and necessitates carefully designed, ontology-aware transformation logic to ensure consistency, scalability, and reproducibility. Thus, both defining a semantic vocabulary to describe VCF file contents and designing a computational conversion process that accurately assigns semantic IRIs to VCF file data fields are non-trivial challenges.

Prior semantic genomics efforts provide important foundations but do not yet offer a general, reproducible VCF-to-RDF tooling that comprehensively converts all fields represented in VCF files into an RDF representation.

Existing conversion tools that offer solutions for the VCF-to-RDF conversion process include VCF2RDF within TOGOvar (26), JVarKit VcfToRdf, the ruby-rdf/rdf-vcf library, and the semantic beacons implementation (10). While these efforts demonstrate the feasibility of representing genomic variants in RDF, they typically model only subsets of VCF semantics, provide limited support for genotype and metadata fields, rely on custom parsing logic, or are embedded within specialized infrastructures rather than offering a standalone, reproducible transformation pipeline. Another notable but limited strategy for VCF-to-RDF translation involves manually combining existing ontologies with YAML-based transformation rules (5), as demonstrated in semantic beacon workflows (10). While effective for simple elements, such rule-based approaches struggle to model deeply nested, dataset-specific structures and therefore lack generalizability across diverse VCF datasets.

Along with tools for RDF conversion, a substantial body of ontological work supports semantic modeling of genomic data. The GA4GH variant schema (33) provides a structured representation of human genomic variants, while FALDO (11) standardizes genomic coordinate modeling and the Sequence Ontology (SO) (17) defines controlled vocabularies for sequence features and variant types. Extensions such as GENO and SNPO (17) address genotype and sample-level semantics. Domain-specific resources including TOGOvar (26), HERO (25), and the SPHN multi-omics schema (32) further enrich variant representation, particularly for clinically oriented applications. Collectively, these vocabularies model important facets of VCF data but do not themselves provide a comprehensive and reproducible transformation mechanism from arbitrary VCF files.

Other large-scale RDF integration initiatives, including Bio2RDF (8) and OpenBioLink (12), demonstrate the potential of knowledge graph approaches for biomedical data integration, including variant-related information. However, their focus lies in curated database integration or downstream learning tasks rather than full semantic coverage of heterogeneous VCF constructs. Across prior efforts, challenges remain regarding completeness, extensibility, tooling robustness, and support for the flexible structures found in INFO, FORMAT, GENOTYPE, and HEADER fields.

Given the heterogeneity of VCF file structure and the need for reproducible, extensible transformation logic, declarative mapping languages provide a principled strategy for VCF-to-RDF conversion. Rather than embedding dataset-specific parsing rules in custom code, mapping languages allow conversion logic to be expressed as reusable, transparent rules that can be shared and adapted across workflows.

In the larger semantic web technology space, several declarative frameworks have been developed for mapping heterogeneous or semi-structured data to RDF. Early approaches include TARQL (2), which supports SPARQL-based transformations of tabular data, and tools such as SDM-RDFizer (22), which prioritize high-performance ingestion of large datasets. For semi-structured data RML (RDF Mapping Language) (16), an extension of R2RML (15), supports non-relational and semi-structured data source conversion, making it well suited to the irregular and nested structure of VCF files. Its ecosystem includes execution engines such as rmlmapper (20) and RMLStreamer (27), supporting both local and scalable processing environments.

RML's design makes it particularly suitable for the VCF format, whose INFO, FORMAT, GENOTYPE and HEADER fields vary in structure and cardinality across files and datasets. Its rule-based, declarative mapping model allows users to encode domain semantics explicitly while adapting to the heterogeneous structure of genomic data, and its ecosystem—including rmlmapper and the distributed engine RMLStreamer—aligns well with FAIR data workflows and scalable transformation pipelines. Among existing tools, RML provides a uniquely extensible approach for modeling the semi-structured and complex fields present in VCF, offering a natural path toward generating reusable semantic representations.

A well-recognized trade-off of RDF representation is semantic inflation: the increase in storage size that results when compact, semi-structured formats such as VCF are expanded into explicit subject–predicate–object triples. In our benchmark, this expansion was substantial: raw RDF outputs were approximately 30.8–80.4× larger than the corresponding input `.vcf.gz` files (Table 1). This expansion is amplified by the use of fully qualified IRIs and the materialization of implicit relationships as explicit graph edges (14).

RDF can be serialized in formats such as N-Triples or N-Quads (13), which are simple, line-oriented representations well suited for streaming and large-scale processing. However, their verbosity makes compression essential for practical storage. General-purpose compressors, more useful for long-term storage, such as gzip routinely reduce RDF file sizes by more than 90%, while Brotli (6) often achieves higher compression ratios with modest additional computational overhead. Beyond byte-level compression, HDT (Header–Dictionary–Triples) (18) provides a domain-specific binary representation of RDF that incorporates dictionary encoding and indexing structures. HDT not only substantially reduces storage requirements but also enables query execution directly over the compressed representation, avoiding full decompression. For archival scenarios, HDT files may be further compressed using gzip or Brotli to maximize storage efficiency. Together, these complementary strategies can reduce the storage overhead introduced by semantic inflation while preserving the interoperability and query capabilities that motivate RDF-based genomic data representation.

In this study, we aim to provide an end-to-end, reproducible command-line pathway from VCF to serialized, compressed RDF through the python-based tool VCF-RDFizer: `https://github.com/ecrum19/VCF-RDFizer`. Specifically, we (i) define a public VCF-RDFizer vocabulary to sematically define VCF file entry data that is aligned with established ontologies: `https://w3id.org/vcf-rdfizer/vocab#`, (ii) implement customizable RML conversion mappings that preserve variant-, genotype-, and metadata-level information, and (iii) evaluate conversion performance of nine public VCF files using VCF-RDFizer, including runtime, memory, output scale, and compression behavior: `https://github.com/ecrum19/vcf-rdfizer-testing`.

VCF-RDFizer CLI Tool

VCF-RDFizer is a command-line workflow that converts input VCF files into semantically enriched RDF using a reproducible mapping pipeline. The workflow integrates parsing, mapping, validation, and optional compression in one execution path. An overview is shown in Figure 1.

Semantic Model

The semantic model combines a dedicated VCF-RDFizer vocabulary (`https://w3id.org/vcf-rdfizer/vocab#`) with established ontologies to maximize interoperability and reuse. Genomic coordinates are represented with FALDO (11), variant typing aligns with the Sequence Ontology (17), and genotype/sample constructs are aligned with existing genomics vocabularies (including HERO where applicable) (25). This hybrid strategy retains VCF-specific semantics while avoiding duplication of mature ontology terms.

The model preserves three information layers: variant-level assertions, genotype/sample assertions, and metadata/header assertions. Core entities are minted using deterministic IRI templates derived from stable VCF attributes (e.g., chromosome, position, reference allele, alternate allele), which keeps identifiers reproducible across runs. When provenance or context separation is required, statements can be organized into named graphs while maintaining a unified SPARQL query surface.

Tool Workflow

VCF-RDFizer is implemented as a CLI pipeline with four stages: (1) VCF conversion to TSV, (2) TSV conversion to RDF via RML mappings and RMLStreamer, (3) RDF validation and optional output aggregation, and (4) optional compression and run statistics outputs. The pipeline preserves metadata and genotype content rather than discarding it during preprocessing. Outputs are emitted as N-Triples by default, with various optional arguments to mediate formatting and/or compression.

Mappings are authored in Turtle as modular RML rule sets that cover fixed VCF columns, variable INFO attributes, FORMAT/genotype structures, and metadata declarations. This modularity allows rules to be extended per annotation family without rewriting the full pipeline. Subject generation, predicate-object assertions, datatypes, and graph targets are declared explicitly in RML for transparent transformation behavior.

To execute conversion using the mappings, we utilized a Java-based tool from the RML ecosystem: RMLStreamer (27). RMLStreamer (version 2.5.0) was executed using its standalone command-line interface (RMLStreamer-v2.5.0-standalone.jar) with default Java heap settings.

Compression formats offered: gzip-compression (via the standard Linux gzip utility), brotli-compression (6) (version v1.1.0), and HDT, using the HDT-c++ command-line tool (18) (version v1.3.3). For both transformations, we tracked the same performance indicators—user time, wall time, input size, and output size—to enable direct comparison of compression effectiveness and computational overhead. For the brotli compression approach, we used a compression level of 7 out of 11 because preliminary experiments with a higher compression level introduced run times of ¿30 minutes. Future work will be done to document the trade-offs for higher levels of brotli compression.

The implementation and mappings are available at `https://github.com/ecrum19/VCF-RDFizer`.

From a user-experience perspective, each tagged VCF-RDFizer version is published across PyPI, Conda-Forge, and DockerHub so users can choose pip-, conda-, or container-based installation with matched versions. The GitHub README serves as the primary user guide, including quick-start commands, end-to-end usage examples, command-line option documentation, and troubleshooting notes. For maintainability, the project follows a release-oriented workflow with versioned changelogs, public issue triage, and planned periodic dependency/container updates to preserve reproducibility over time.

Tool Testing

VCF Test Data

For testing our workflow, we used one publicly available VCF file from the NIST Genome in a Bottle project – 0GOOR_HG002 (4) and 5 publicly available VCF files from the Personal Genome Project (1). The selected files vary in size, record counts, and annotation patterns, and were chosen to stress realistic differences in metadata and field content while remaining standards-compliant with VCF specifications (3). For more information or to replicate all tests performed, see `https://github.com/ecrum19/vcf-rdfizer-testing`.

Testing Environment

VCF-RDFizer automatically logs wall time, CPU time, peak memory (via resident set size (RSS)), output directory size, and total

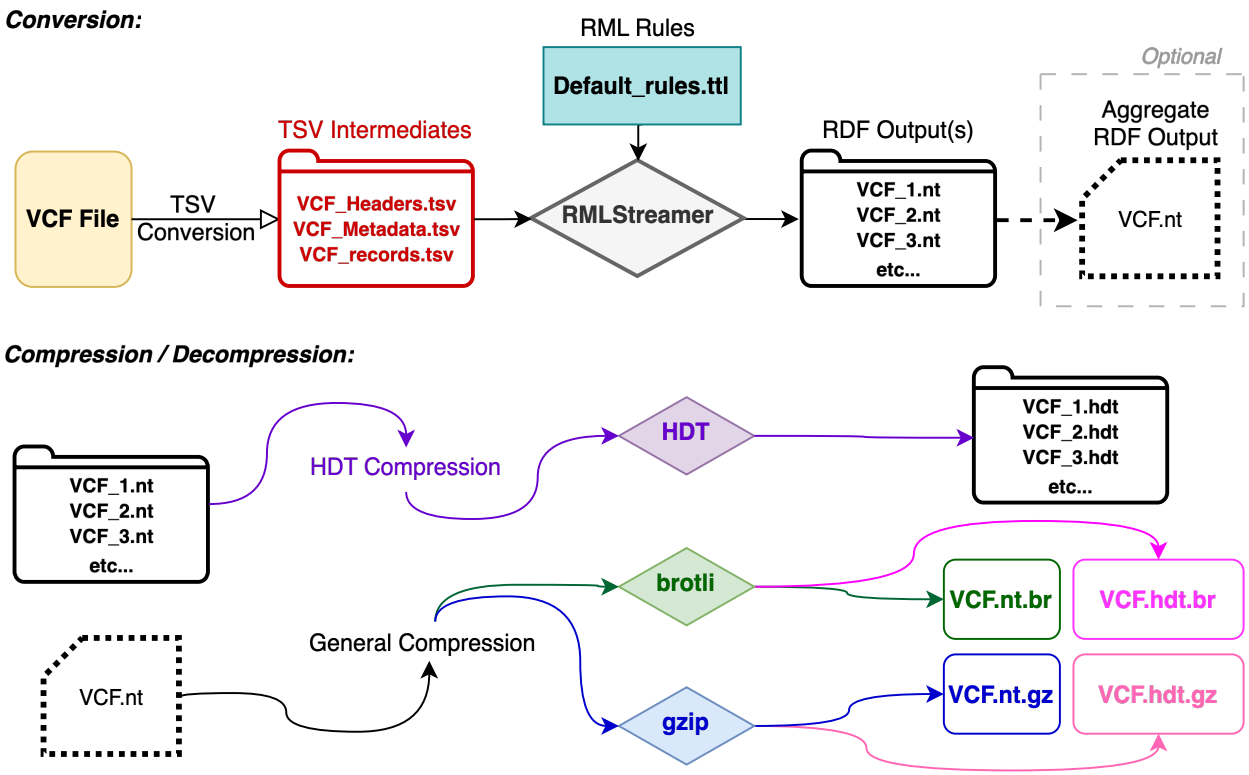

Figure 1 VCF-RDFizer Pipeline Workflow The flowchart depicts end-to-end conversion from VCF input to semantically enriched, compressed RDF outputs. **Conversion**: VCF-RDFizer parses converts VCF file into CSV intermediates, uses RML mappings and RMLStreamer to generate N-Triples, optionally aggregates VCF RDF outputs to a single .nt file. **Compression**: VCF-RDFizer compresses RDF output(s) via HDT and/or General Compression (gzip, Brotli) for downstream storage and querying. HDT compressed RDF files can also be further compressed via general compression (either gzip or Brotli). **Decompression**: All steps in the compression workflow are reversible.

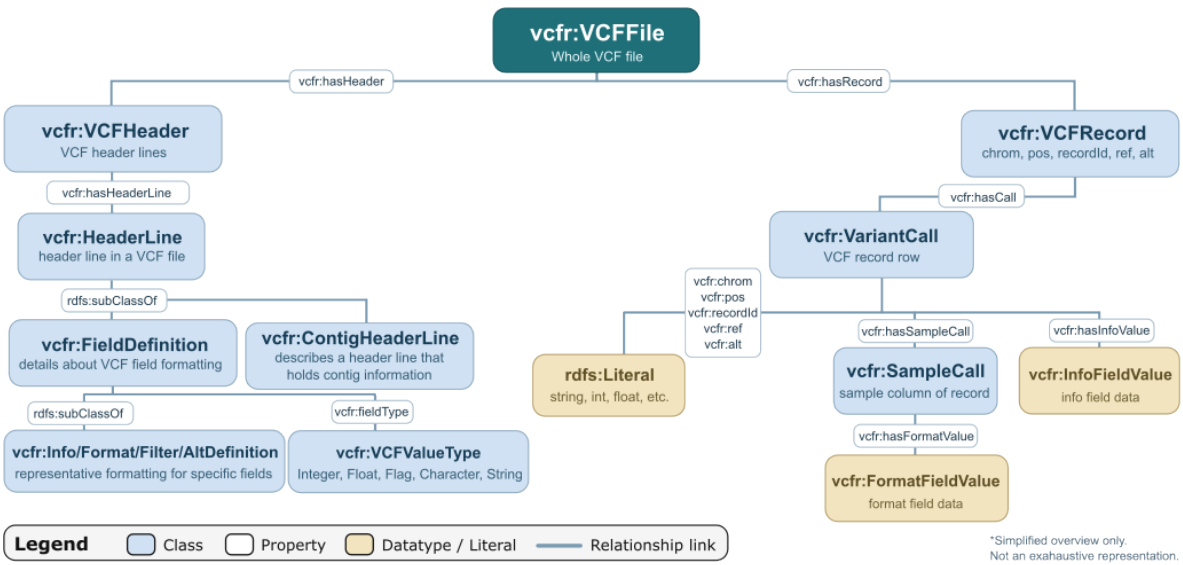

Figure 2 Semantic VCF Vocabulary (sample) The figure depicts a simplified representation of the semantic heirarchy for representation of the data within a VCF file. All prefixes *vcfr:* are an abbreviation for `https://w3id.org/vcf-rdfizer/vocab#` . A companion website was created for a more complete, detailed, and interactive representation of this vocabulary. Website available at: `https://ecrum19.github.io/VCF-RDFizer-vocabulary/`.

Input VCF	Input .vcf.gz size (MB)	Wall time (s)	Max RSS (MB)	Output raw RDF size (MB)	# triples
0GOOR_HG002	387.9	621.0	1,602.4	31,181.3	167,958,298
60820188474283	1,054.8	719.0	1,674.0	35,408.1	185,443,019
60820188475559	1,233.0	872.2	1,546.3	37,955.6	198,039,339

Table 1 Conversion statistics per input file. Sizes are decimal MB.

triple counts to support reproducible performance comparisons across datasets and hardware setups. All metrics are collected via bash scripts at tool run-time to ensure reproducibility and consistent measurement across all experiments. Validation included syntax checks and targeted inspection of variant, genotype, and metadata nodes to verify that semantic links and datatypes were generated as intended. The full set of recorded metrics can be found in the test's dedicated git repo: `https://github.com/ecrum19/vcf-rdfizer-testing`.

All performance experiments were conducted on a workstation running Ubuntu 24.04.4 LTS with an Intel(R) Xeon(R) Silver 4309Y CPU @ 2.80GHz (8 cores, 2.8 GHz), 33.6 GB RAM and a 210 GB SSD, Kernel version 6.8.0-45-generic. Software versions: Docker 28.2.2, Python 3.12.3.

Results

VCF-RDFizer - General Performance Observations

Table 1 summarizes conversion performance across three representative VCF datasets. Input sizes ranged from 387.9–1233.0 MB and produced 31,181.3–37,955.6 MB of raw RDF, corresponding to semantic expansion factors of 30.8×–80.4× relative to compressed VCF input. The converted outputs contained 167.96–198.04 million triples, indicating that the mapping workflow scaled consistently across the tested files.

RMLStreamer successfully completed conversion for all three datasets. Wall-clock conversion time ranged from 621.0 to 872.2 seconds (10.4–14.5 minutes), while peak RSS remained within 1546.3–1674.0 MB. These measurements indicate that conversion is tractable on commodity workstation hardware, although memory pressure is non-negligible.

Figure 3 complements Table 1 by showing size, wall-time, and memory trends for conversion and downstream compression. Overall, the conversion stage dominates end-to-end runtime, while compression methods primarily trade additional time for reduced storage footprint.

VCF TSV Intermediates

VCF inputs were transformed into TSV intermediates to provide a compatible logical source for RMLStreamer. This preprocessing enabled consistent field-level extraction and stable downstream mapping execution across all tested datasets.

RMLStreamer RDF Generation Performance

For each dataset, RMLStreamer emitted eight partitioned RDF output files. Partition-level triple counts were well balanced (e.g., 20.91–21.08 million for 0GOOR_HG002), indicating stable parallel execution behavior during conversion.

Data Compression

Compression outcomes are summarized in Table 2. Across all datasets, every method substantially reduced raw RDF size, with clear trade-offs between compression ratio, runtime, and memory usage.

HDT Compression

HDT reduced raw RDF outputs to 1887.4–2102.6 MB, corresponding to a 93.26–94.68% reduction versus uncompressed RDF. HDT runtime ranged from 205.0 to 227.0 seconds, and HDT was the most memory-intensive compression step with peak RSS of 2610.0–2950.0 MB. This behavior is consistent with HDT's dictionary and index construction overhead.

Gzip Compression

Gzip reduced RDF size to 883.2–1271.7 MB (96.64–97.17% reduction). Gzip wall time was 187.1–245.9 seconds and peak RSS was consistently 4.0 MB. These results confirm gzip as a low-memory, low-friction archival compression baseline.

Brotli Compression

Brotli achieved stronger compression than gzip for all datasets, producing 480.9–775.8 MB outputs (97.93–98.46% reduction). This gain came at higher runtime: 274.6–362.7 seconds. Peak RSS remained modest at 47.0–50.0 MB, indicating that Brotli's additional cost is primarily CPU time rather than memory footprint.

Overall, Brotli offers a compelling trade-off: significantly better compression ratios at the cost of moderately longer runtimes, without imposing meaningful memory requirements. As with gzip, Brotli does not improve query performance or alter the structure of the RDF data, and thus serves primarily as a highly efficient archival format.

Compound Compression

Applying general-purpose compression after HDT yielded the smallest artifacts in this study. HDT+gzip reduced outputs to 215.8–371.0 MB (98.99–99.31% reduction versus raw RDF) with an additional 24.5–38.0 seconds of runtime and negligible memory overhead (∼4 MB RSS). HDT+brotli achieved the strongest compression overall at 182.0–318.9 MB (99.16–99.42% reduction), with additional runtime of 60.1–92.6 seconds and 99.3–102.4 MB peak RSS. Thus, HDT+gzip provided the fastest secondary compression, while HDT+brotli consistently produced the smallest final files.

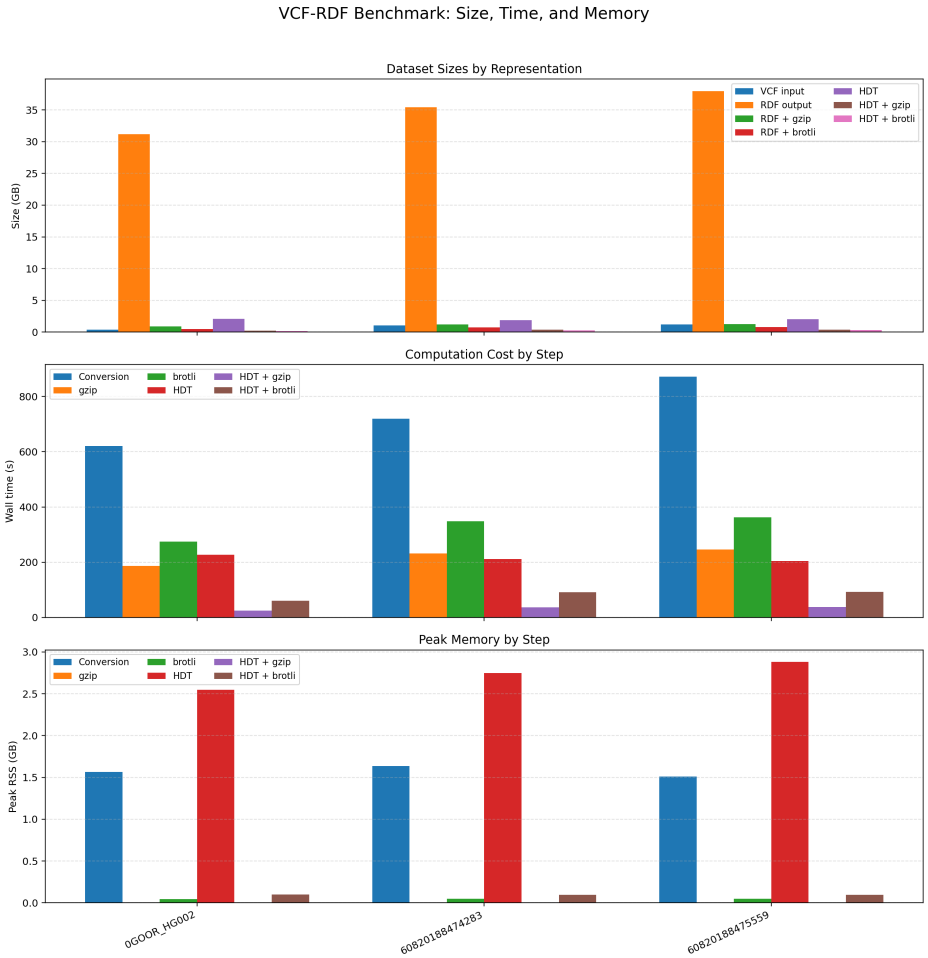

Figure 3 Full Mode VCF Conversion to RDF - Metrics Comparison of output sizes, computation times, and memory usage for converting three VCF datasets into RDF and compressed derivatives. The top panel shows resulting file sizes for input VCF, raw RDF, and each compression variant. The middle panel reports wall-clock execution times for conversion and compression stages. The bottom panel presents peak resident memory (RSS) across the same stages. Together, these measurements quantify the computational and storage trade-offs of semantic lifting from VCF to RDF.

Input file	Time (s)					Size (MB)				
	HDT	Gzip	Brotli	HDT+Gz	HDT+Br	HDT	Gzip	Brotli	HDT+Gz	HDT+Br
0GOOR_HG002	227.0	187.1	274.6	24.5	60.1	2,102.6	883.2	480.9	215.8	182.0
60820188474283	212.0	231.6	347.9	36.9	91.8	1,887.4	1,189.2	731.4	357.6	291.6
60820188475559	205.0	245.9	362.7	38.0	92.6	2,020.6	1,271.7	775.8	371.0	318.9

Table 2 Compression statistics per input file. Times are wall-clock seconds; sizes are decimal MB.

Discussion

Research Software Publishing Practices

To align with established research software publishing guidance (23), we explicitly address four practical publication dimensions. First, the source code is developed and versioned in a public repository from the first release. Second, discoverability and metadata exposure are supported through synchronized releases on GitHub, PyPI, Conda-Forge, and DockerHub. Third, reuse conditions are made explicit through an OSI-approved open-source license and clear citation instructions. Fourth, community/governance transparency is handled through public issue tracking, release notes, and a documented maintenance plan. Together, these practices complement the technical results by ensuring that users can discover, install, cite, and sustain the software in reproducible environments.

Implications for Users Running Conversion

The results indicate that VCF-to-RDF conversion is practical for single-sample files of the tested scale: 387.9–1233.0 MB inputs were converted in 10.4–14.5 minutes with peak RSS of approximately 1.5–1.7 GB. For users, this means the primary operational planning variable is memory headroom per concurrent conversion job.

A comprehensive semantic lifting of VCF into RDF requires addressing both structural heterogeneity and gaps in existing genomic ontologies. While several well-established ontologies model important aspects of genomic variation—including FALDO for genomic coordinates, the Sequence Ontology (SO) for variant types, GENO for genotype modeling, and HERO for clinically oriented genomic semantics—no single ontology currently provides complete coverage of the VCF specification. In particular, the VCF HEADER section and the highly variable INFO, FORMAT, and GENOTYPE fields expose semantic constructs that are only partially represented across existing vocabularies.

The VCF format is formally specified, yet it is intentionally flexible. INFO and FORMAT fields are extensible and frequently dataset-specific; metadata declarations in HEADER lines define field semantics dynamically; and alternate allele representations range from simple SNPs to complex structural variants. Moreover, differences across VCF specification versions introduce subtle but meaningful changes in header conventions and field definitions. These characteristics complicate the creation of a stable, ontology-driven mapping model capable of uniformly covering arbitrary VCF files.

To address these challenges, we developed the publicly available VCF-RDFizer vocabulary (https://w3id.org/vcf-rdfizer/vocab#), which provides a lightweight semantic layer explicitly aligned with the VCF specification while remaining interoperable with established genomic ontologies. Rather than attempting to replace domain ontologies such as FALDO or SO, the vocabulary acts as a structural scaffold: it models core VCF entities (e.g., Variant, Allele, Genotype, Sample, HeaderField) and their relationships in a manner faithful to the file format, while enabling alignment to external ontological terms where appropriate. This approach separates format-level semantics from biological domain semantics, allowing transformation logic to remain stable even when downstream ontological commitments evolve.

The modeling challenge becomes substantially more complex in multi-sample VCF files, where per-sample FORMAT fields must be associated with both variant-level and sample-level identifiers. Maintaining consistent and reproducible IRI schemes across variants, samples, and genotypes is essential for ensuring referential integrity in the resulting RDF graph. Although RML mappings can express such relationships, they require carefully designed rule sets that accommodate dataset-specific variability. Our evaluation focused on single-sample VCF files to establish baseline computational feasibility; further work is needed to evaluate semantic completeness and performance across multi-sample and clinically annotated VCF datasets.

An additional practical limitation of the current implementation is the preprocessing step that transforms VCF into a simplified tabular representation for compatibility with the RML ecosystem. Because VCF is not natively supported as a logical source in RML, this preprocessing step omits HEADER metadata in the present study. A more robust future implementation would define VCF as a custom logical source within the RML framework, enabling direct modeling of header-defined semantics and eliminating information loss during transformation.

The semantic challenges identified here highlight that VCF-to-RDF conversion is not merely a syntactic transformation but a modeling exercise requiring explicit design decisions regarding identifier stability, ontology alignment, and representation of heterogeneous field structures. The VCF-RDFizer vocabulary represents a first step toward a reproducible and extensible semantic framework capable of supporting full VCF coverage in future work.

Our evaluation demonstrates that RML provides a functional and principled mechanism for converting VCF data into RDF. The declarative mapping model allows transformation logic to be expressed explicitly and reproducibly, separating semantic intent from procedural parsing code. This architectural separation is a key advantage: mappings can be versioned, inspected, extended, and reused across datasets without modifying the underlying execution engine.

At the same time, our results indicate that general-purpose RML engines such as RMLStreamer incur non-trivial computational and memory overhead. Even when mapping a restricted subset of VCF fields, memory consumption approached 2 GB and execution time scaled proportionally with input size. These observations suggest that while RML is suitable for semantic lifting of VCF data, its current execution model is not yet optimized for the structural characteristics of large genomic files.

The compression results provide a clear post-conversion decision path. If the priority is fast, low-overhead archival compression, gzip is the simplest default (96.64–97.17% reduction, 4 MB RSS). If maximizing size reduction is more important than runtime, brotli improves compression (97.93–98.46%) at higher wall time. If downstream querying is required, HDT remains the key format despite higher build-time memory use (2.6–3.0 GB RSS), and can be further compressed with gzip or brotli for long-term storage.

HDT (Header–Dictionary–Triples) provides one of the most compelling compression approaches for RDF data, particularly for its foundation for efficient querying of the compressed data. Unlike traditional compression formats that simply compress bytes, HDT restructures RDF into a binary, indexed format consisting of a header, a dictionary of terms, and a highly compact triple component. This structure not only achieves significant size reduction—often surpassing general-purpose compressors for triple-heavy datasets—but also enables efficient querying without requiring full decompression. HDT supports fast triple pattern lookups and can be used directly with HDT-aware SPARQL engines (31), making it well-suited for exploratory workflows, federated querying, or HTTP-mediated variant queries. HDT's ability to combine compression with optimizations that encourage queryability represents a major advantage for long-term management of semantic VCF data. Another alternative semantic compression approach that functions similarly to HDT is COTTAS (7) and could be explored in future work.

Implications for RDF Genomic Workflows

Beyond conversion feasibility, the main result is that VCF data can be lifted into a graph representation at a cost profile that is manageable for routine workflows. This matters because RDF enables semantic querying over variant, genotype, sample, and annotation entities, and HDT-based representations can support query-oriented access patterns in compressed form (31).

The same representation also improves data linking potential. Stable IRIs and ontology-aligned predicates provide a direct mechanism to connect variant-level observations with external biomedical resources (e.g., phenotype and pharmacogenomic knowledge bases), reducing repeated format-specific ETL effort.

A third implication concerns governance. RDF makes it possible to attach policy metadata to fine-grained entities or named graph

slices, enabling selective disclosure patterns rather than whole-file sharing. This creates a practical path toward policy-aware genomic data services using standards-aligned mechanisms (e.g., ODRL/Web access-control style models) (9).

Impact and Future Work

The results presented in this study establish the technical feasibility of converting VCF data into RDF using declarative semantic mappings. Beyond feasibility, however, the broader impact of this work lies in the new representational possibilities enabled by expressing genomic variant data as structured knowledge graphs.

Representing VCF data in RDF enables explicit modeling of relationships between variants, samples, genotypes, annotations, and external biomedical resources. Unlike traditional file-centric workflows, graph-based representations allow variant data to be directly linked to phenotype databases, pharmacogenomic resources, curated knowledge bases, and governance metadata using shared identifiers and ontological terms. This creates a foundation for integrative querying across heterogeneous datasets using standardized query languages such as SPARQL, potentially reducing reliance on format-specific tooling and ad hoc parsing pipelines.

In clinical and translational contexts, semantic representations may further support fine-grained access control and selective data disclosure. By associating variant-level or genotype-level resources with policy annotations expressed through web standards such as ODRL or Web Access Control mechanisms, RDF-based infrastructures could enable controlled exposure of genomic information without distributing complete VCF files. While such governance frameworks were not implemented in this study, the feasibility results presented here provide an essential prerequisite for exploring these privacy-preserving architectures.

Several concrete directions for future work emerge from this study. First, we aim to refine the semantic coverage of complex VCF fields, i.e. INFO, FORMAT, GENOTYPE, and HEADER fields, and reassess our converison memory strategies to support large VCF files and multi-sample VCF files. An interesting possibility to reduce memory complexity is presented by RDF triple compression at time of generation using a framework like Jelly(30). Second, we plan to eliminate the current preprocessing step by implementing VCF as a native logical source within the RML ecosystem, thereby preserving header metadata and simplifying workflow integration. Third, we intend to evaluate SPARQL query performance over serialized and HDT-compressed RDF outputs to assess the practical implications of RDF-based querying at genomic scale.

Big picture, we aim to investigate domain-specific optimizations for large-scale VCF-to-RDF conversion, including streaming-aware execution strategies and reusable mapping templates tailored to common VCF patterns. While highly theoretical, an intriguing solution could be performing RDF conversions on-the-fly in response to a specific query need or serializing the RDF output into multiple named graphs or chunks, such as per-chromosome, per-sample, or per-annotation-type graphs, which could offer powerful, efficient use-case motivated VCF representation strategies. This form of logical partitioning could significantly improve usability and query performance in downstream workflows, allowing systems to load or index only the relevant portions of the data. Utilizing unique named graphs could also help organize provenance, track transformations, and support distributed storage strategies for very large genomic datasets.

Ultimately, a compelling prospective technology that could leverage the semantic representation of VCF data is a genome-data API layered on top of the VCF RDF representations. Because RDF inherently supports fine-grained linking and entity-level access, such an API could allow users or services to request only specific variants or annotations and receive them conditionally—based on access rights, consent, or embedded data-governance rules. This would offer a stark contrast to traditional VCF sharing, which is typically an all-or-nothing process, and could enable more privacy-aware genomic data dissemination.

To translate these results into production infrastructures, future evaluations should also test end-to-end policy-aware API scenarios (query + authorization + selective response), since this is where semantic querying, data linking, and privacy enforcement intersect most directly for real users.

Together, these efforts position VCF-RDFizer not merely as a conversion tool, but as a step toward semantically interoperable genomic data infrastructures that support scalable integration, expressive querying, and governance-aware data sharing. As more comprehensive semantic models and more advanced mapping rules are developed, the resulting knowledge graphs will become increasingly suitable for sophisticated query-driven workflows, privacy-aware data services, and large-scale genomic data integration.

Conflicts of interest

The authors declare that they have no competing interests.

Funding

This work is supported in part by funds from the Research Foundation – Flanders (FWO) (SB Fellowship 1S27825N). Ruben Taelman is a postdoctoral fellow of the Research Foundation – Flanders (FWO) (1202124N).

Data availability

The data underlying this article are available in a git repository at `https://github.com/ecrum19/vcf-rdfizer-testing`.

Author contributions statement

E.C. and G.E. conceived the tool and the experiments. E.C. implemented the conversion workflow and executed the experiments. E.C. and R.T. analyzed the results. E.C. wrote the manuscript draft. E.C., R.T., G.E., and B.B. reviewed and edited the manuscript.

Acknowledgments

The authors thank the anonymous reviewers for their valuable suggestions.

References

1. Personal Genome Project, 2005.

2. Tarql: SPARQL for Tables – Tarql – SPARQL for Tables: Turn CSV into RDF using SPARQL syntax, 2019.
3. The Variant Call Format Specification, October 2024.
4. NIST Genome in a Bottle, November 2025.
5. yaml/yaml-spec, November 2025. original-date: 2009-04-01T20:52:05Z.
6. Jyrki Alakuijala, Andrea Farruggia, Paolo Ferragina, Eugene Kliuchnikov, Robert Obryk, Zoltan Szabadka, and Lode Vandevenne. Brotli: A General-Purpose Data Compressor. *ACM Transactions on Information Systems*, 37(1):1–30, January 2019.
7. Julián Arenas-Guerrero and Sebastián Ferrada. Cottas: Columnar triple table storage for efficient and compressed rdf management. In *The Semantic Web – ISWC 2025*. Springer, 2025.
8. François Belleau, Marc-Alexandre Nolin, Nicole Tourigny, Philippe Rigault, and Jean Morissette. Bio2RDF: Towards a mashup to build bioinformatics knowledge systems. *Journal of Biomedical Informatics*, 41(5):706–716, October 2008.
9. Tim Berners-Lee, Henry Story, and Sarven Capadisli. Web Access Control, March 2021.
10. Alexandrina Bodrug-Schepers, Hugo Chabane, Gabriela Montoya, Richard Redon, Alban Gaignard, and Patricia Serrano-Alvarado. Semantic Beacons: a framework to support federated querying over genomic variants and public Knowledge Graphs. Barcelona, Spain, February 2025. CEUR Workshop Proceedings.
11. Jerven T. Bolleman, Christopher J. Mungall, Francesco Strozzi, Joachim Baran, Michel Dumontier, Raoul J. P. Bonnal, Robert Buels, Robert Hoehndorf, Takatomo Fujisawa, Toshiaki Katayama, and Peter J. A. Cock. FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation. *Journal of Biomedical Semantics*, 7(1):39, December 2016.
12. Anna Breit, Simon Ott, Asan Agibetov, and Matthias Samwald. OpenBioLink: A benchmarking framework for large-scale biomedical link prediction. 2019. Publisher: arXiv Version Number: 2.
13. Gavin Carothers. RDF 1.1 N-Quads, February 2014.
14. Richard Cyganiak, David Wood, and Markus Lanthaler. Rdf 1.1: Concepts and abstract syntax, February 2014.
15. Souripriya Das, Seema Sundara, and Richard Cyganiak. R2RML: RDB to RDF Mapping Language, September 2012.
16. Ben De Meester, Pieter Heyvaert, and Thomas Delva. RDF Mapping Language (RML), June 2024.
17. Karen Eilbeck, Suzanna E Lewis, Christopher J Mungall, Mark Yandell, Lincoln Stein, Richard Durbin, and Michael Ashburner. The Sequence Ontology: a tool for the unification of genome annotations. *Genome Biology*, 6(5):R44, April 2005.
18. Javier D. Fernández, Miguel A. Martínez-Prieto, Claudio Gutiérrez, Axel Polleres, and Mario Arias. Binary rdf representation for publication and exchange (hdt). *Web Semantics: Science, Services and Agents on the World Wide Web*, 19:22–41, 2013.
19. Steve Harris, Andy Seaborne, and Eric Prud'hommeaux. SPARQL 1.1 query language, March 2013.
20. Pieter Heyvaert, Dylan Van Assche, Ben De Meester, Gerald Haesendonck, Els de Vleeschauwer, and Sitt Min Oo. RMLMapper-JAVA, September 2025.
21. Renato Iannella and Serena Villata. ODRL Information Model 2.2, February 2018.
22. Enrique Iglesias, Samaneh Jozashoori, David Chaves-Fraga, Diego Collarana, and Maria-Esther Vidal. SDM-RDFizer: An RML Interpreter for the Efficient Creation of RDF Knowledge Graphs. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pages 3039–3046, Virtual Event Ireland, October 2020. ACM.
23. Rafael C. Jiménez, Mateusz Kuzak, Monther Alhamdoosh, Michelle Barker, Bérénice Batut, Marc Borg, Salvador Capella-Gutiérrez, Neil P. Chue Hong, Martin Cook, Manuel Corpas, Michael Flannery, Leyla Garcia, Josep L. Gelpí, Simon Gladman, Carole Goble, Miguel González Ferreiro, Alejandra Gonzalez-Beltran, Philip C. Griffin, Björn Grüning, Jon Hagberg, Petr Holub, Rob Hooft, Jon Ison, Sarah Jones, Florian Kaiser, Sunghwan Kim, Matthew Krafczyk, Anna Krystalli, Hilmar Lapp, Ann Lonie, Céline Marek, Michael P. Mazanetz, Aleksandra Nenadic, Wolfgang Petersen, Esther Plomp, Mikhail Pomazkov, Karthik Ram, Stian Soiland-Reyes, Alexey Sokolov, Andreas Struck, Radoslaw Suchecki, Rutger Vos, Alan R. Williams, Patrick Brack, Marta W. Muelas, Alexander D. Diehl, Jean-Baptiste Poline, Teresa K. Atwood, Alexander Mansurov, Larisa N. Soldatova, Carl Kingsford, Paul J. Kersey, Mark D. McDowall, Magne Østeras, Cristina Guijarro-Clarke, and Steve Crouch. Four simple recommendations to encourage best practices in research software. *F1000Research*, 6:876, 2017.
24. H. L. McLeod. Cancer pharmacogenomics: Early promise, but concerted effort needed. 339(6127):1563–1566, March 2013.
25. Laura Menotti, Mirco Cazzaro, Manuel Rueda, Ivo G Gut, and Gianmaria Silvello. HERO-Genomics: Bridging Genomic Data and Ontological Modelling. Barcelona, Spain, February 2025. CEUR Workshop Proceedings.
26. Nobutaka Mitsuhashi, Licht Toyo-oka, Toshiaki Katayama, Minae Kawashima, Shuichi Kawashima, Kazunori Miyazaki, and Toshihisa Takagi. TogoVar: A comprehensive Japanese genetic variation database. *Human Genome Variation*, 9(1):44, December 2022.
27. Sitt Min Oo, Gerald Haesendonck, Ben De Meester, and Anastasia Dimou. RMLStreamer-SISO: An RDF Stream Generator from Streaming Heterogeneous Data. In Ulrike Sattler, Aidan Hogan, Maria Keet, Valentina Presutti, João Paulo A. Almeida, Hideaki Takeda, Pierre Monnin, Giuseppe Pirrò, and Claudia d'Amato, editors, *The Semantic Web – ISWC 2022*, volume 13489, pages 697–713. Springer International Publishing, Cham, 2022. Series Title: Lecture Notes in Computer Science.
28. SIB Swiss Institute of Bioinformatics RDF Group Members, Adrian Altenhoff, Amos Bairoch, Parit Bansal, Delphine Baratin, Frederic Bastian, Jerven Bolleman*, Alan Bridge, Frédéric Burdet, Katrin Crameri, Jérôme Dauvillier, Christophe Dessimoz, Sebastien Gehant, Natasha Glover, Kristin Gnodtke, Catherine Hayes, Mark Ibberson, Evgenia Kriventseva, Dmitry Kuznetsov, Lisacek Frédérique, Florence Mehl, Tarcisio Mendes De Farias*, Pierre-André Michel, Sébastien Moretti, Anne Morgat, Sabine Österle, Marco Pagni, Nicole Redaschi, Marc Robinson-Rechavi, Kasun Samarasinghe, Ana-Claudia Sima, Damian

Szklarczyk, Orlin Topalov, Vasundra Touré, Deepak Unni, Christian Von Mering, Julien Wollbrett, Monique Zahn-Zabal*, and Evgeny Zdobnov. The SIB Swiss Institute of Bioinformatics Semantic Web of data. *Nucleic Acids Research*, 52(D1):D44–D51, January 2024.

29. Erika Souche, Sergi Beltran, Erwin Brosens, John W. Belmont, Magdalena Fossum, Olaf Riess, Christian Gilissen, Amin Ardeshirdavani, Gunnar Houge, Marielle Van Gijn, Jill Clayton-Smith, Matthis Synofzik, Nicole De Leeuw, Zandra C. Deans, Yasemin Dincer, Sebastian H. Eck, Saskia Van Der Crabben, Meena Balasubramanian, Holm Graessner, Marc Sturm, Helen Firth, Alessandra Ferlini, Rima Nabbout, Elfride De Baere, Thomas Liehr, Milan Macek, Gert Matthijs, Hans Scheffer, Peter Bauer, Helger G. Yntema, and Marjan M. Weiss. Recommendations for whole genome sequencing in diagnostics for rare diseases. 30(9):1017–1021, September 2022.
30. Piotr Sowiński, Karolina Bogacka, Anastasiya Danilenka, and Nikita Kozlov. Jelly: a fast and convenient rdf serialization format. In *Joint Proceedings of Posters, Demos, Workshops, and Tutorials of the 21st International Conference on Semantic Systems co-located with 21st International Conference on Semantic Systems (SEMANTiCS 2025), Vienna, Austria, September 3-5, 2025*, volume 4064 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2025.
31. Ruben Taelman, Joachim Van Herwegen, Miel Vander Sande, and Ruben Verborgh. Comunica: A modular SPARQL query engine for the web. In *Proceedings of the 17th International Semantic Web Conference*, pages 239–255. Springer International Publishing, 10 2018.
32. Eelke Van Der Horst, Deepak Unni, Femke Kopmels, Jan Armida, Vasundra Touré, Wouter Franke, Katrin Crameri, Elisa Cirillo, and Sabine Österle. Bridging Clinical and Genomic Knowledge: An Extension of the SPHN RDF Schema for Seamless Integration and FAIRification of Omics Data. December 2023.
33. Alex H. Wagner, Lawrence Babb, Gil Alterovitz, Michael Baudis, Matthew Brush, Daniel L. Cameron, Melissa Cline, Malachi Griffith, Obi L. Griffith, Sarah E. Hunt, David Kreda, Jennifer M. Lee, Stephanie Li, Javier Lopez, Eric Moyer, Tristan Nelson, Ronak Y. Patel, Kevin Riehle, Peter N. Robinson, Shawn Rynearson, Helen Schuilenburg, Kirill Tsukanov, Brian Walsh, Melissa Konopko, Heidi L. Rehm, Andrew D. Yates, Robert R. Freimuth, and Reece K. Hart. The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification. *Cell Genomics*, 1(2):100027, November 2021.